Lab 8: Explained

Lab 8: GitHub Actions, a review gate and a scheduled job

The big idea

Three automated helpers: one answers when called, one checks quality before merging, and one runs a scheduled housekeeping report.

@claude comment	Interactive response in issue/PR
Pull request	Review gate fails on important findings
Nightly schedule	TODO report runs automatically
GitHub secret	Stores API key safely

What you should understand

GitHub Actions can respond to @claude, block a merge using a failing check, and run a scheduled job. Keep API keys in repository secrets, not workflow files.

Lab walkthrough — explained in simple English

Goal: run Claude in GitHub: answer `@claude` comments, block a bad pull request, and run a nightly job. Tool: GitHub + Claude Code GitHub Action · Time: 30 minutes

Before you start

1. A GitHub account, and Git installed (`git --version`). On Windows, install [Git for Windows](https://git-scm.com/downloads/win) if needed.

2. Your Claude API key. In this lab it goes into a GitHub secret (Step 2), not into a file and not into your terminal.

3. Optional: Claude Code installed, if you want to use `/install-github-app` in Step 3.

Files in this folder

File	What it is
`.github/workflows/claude.yml`	Claude answers when someone writes `@claude` in an issue or PR comment
`.github/workflows/review-gate.yml`	On every pull request, runs `review.py`. Important findings fail the check.
`.github/workflows/nightly.yml`	Every night at 03:30 UTC (or on a button press), lists TODOs without a ticket
`review.py`, `spec.md`, `discount.py`, `writer_notes.md`, `CLAUDE.md`	The independent reviewer from Lab 7 and its inputs. `discount.py` has bugs.
`orders.py`, `billing.py`	Code with TODO comments, for the nightly job

Steps

Step 1: Put this folder on GitHub

• On GitHub, create a new private repository (empty, with no README).

• In your terminal:

```
cd lab08-github-actions
git init
git add .
git commit -m "start"
git branch -M main
git remote add origin https://github.com/<you>/<repo>.git
git push -u origin main
```

Step 2: Add your API key as a secret

On GitHub, open your repo and go to Settings → Secrets and variables → Actions → New repository secret.

• Name: `ANTHROPIC_API_KEY`

• Value: your key

Never put the key in a workflow file. Always use a secret.

Step 3: Install the Claude GitHub App

Open https://github.com/apps/claude, click Install, and choose your repository.

(Or start `claude` in this folder, run `/install-github-app` and follow the steps.)

Step 4: Talk to Claude in an issue

• Open the Issues tab and create an issue called `Question`.

• Add a comment:

```
@claude What does billing.py do? Answer in 3 bullet points.
```

 Within a minute or two, Claude replies in the issue. You can watch it run in the Actions tab.

This is interactive mode: the workflow has no `prompt`, so it waits for `@claude`.

Step 5: The review gate blocks a bad pull request

In your terminal:

```
git checkout -b small-change
```

Add a comment line at the top of `discount.py`, for example `# small change`, then:

```
git add .
git commit -m "small change"
git push -u origin small-change
```

On GitHub, open a pull request from `small-change`.

 The Review gate check fails (red). Open the check's log to see the findings: `discount.py` still has its bugs.

Now replace `discount.py` with the fixed version below, then commit and push again:

```
def final_total(subtotal_cents: int, loyalty: bool = False) -> int:
    """Return the final total in integer cents (see spec.md)."""
    if subtotal_cents <= 0:
        raise ValueError("subtotal must be greater than 0")
    if subtotal_cents >= 10000:
        tier = 10
    elif subtotal_cents >= 5000:
        tier = 5
    else:
        tier = 0
    total = subtotal_cents - subtotal_cents * tier // 100
    if loyalty:
        total = total - total * 2 // 100
    return total
```

```
git add .
git commit -m "fix discount rules"
git push
```

 The check passes (green).

Anthropic's managed Code Review (Team and Enterprise plans) posts review comments but never blocks a merge. If you need a gate, build one like this.

Step 6: Run the scheduled job

On GitHub, go to Actions → Nightly TODO report → Run workflow.

 The job runs. Open the log to see the TODO report.

In `nightly.yml`:

• `schedule: cron: "30 3 *"` runs it every day at 03:30 UTC, from the default branch only.

• `workflow_dispatch` adds the Run workflow button, for testing.

• Because it has a `prompt`, it runs in automation mode, with no `@claude` needed.

What you learned

• `anthropics/claude-code-action@v1` with no `prompt` is interactive (`@claude`). With a `prompt`, it runs automatically.

• Keep keys in secrets. Add `timeout-minutes`, and `concurrency` for pull requests.

• Your own review job, with an exit code, can block a merge.

Clean up

Delete the test repository on GitHub when you're done (Settings → Delete this repository).

REMEMBER
GitHub Actions can respond to @claude, block a merge using a failing check, and run a scheduled job. Keep API keys in repository secrets, not workflow files.